OPERATING SYSTEMS ORANGE PROBLEM

Name: Sharvanee Naru

SRN: PES2UG24CS459

SEC: 4-H

1A

```
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B

```
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

2A

```
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ ./test_tree
\Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$
```

2B

```
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ ./pes init
Initialized empty PES repository
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ echo "hello" > file.txt
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ ./pes add file.txt
added file.txt
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ ./pes status
Staged changes:
  staged: file.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  test_tree.c
  commit.h
  test_sequence.sh
  index.c
  object.c
  index.h
  pes.h
  test_objects.c
  tree.h
  tree.c
  commit.c
  pes.c
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776264848 6 file.txt
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$
```

OPERATING SYSTEMS ORANGE PROBLEM

3A

```
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ ./pes init
Initialized empty PES repository
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ echo "hello" > file1.txt
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ echo "world" > file2.txt
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ ./pes add file1.txt file2.txt
added file1.txt
added file2.txt
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ ./pes status
Staged changes:
  staged: file.txt
  staged: file1.txt
  staged: file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  test_tree.c
  commit.h
  test_sequence.sh
  index.c
  object.c
  index.h
  pes.h
  test_objects.c
  tree.h
  tree.c
  commit.c
  pes.c
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$
```

3B

```
  pes.c
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776264848 6 file.txt
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776267524 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776267533 6 file2.txt
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$
```

4A

```
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ ./pes log
commit 70db70c034ff7f8c0e6ae94cc8e54da0e448bd22f6f96bf4e18c15d5fd3a2f5d
Author: PES User <pes@localhost>
Message: Add farewelldd f

commit 01eadfeac178217e2db9a7b399a62a3da57bb9ac313a1032f8a637fed07ec47a
Author: PES User <pes@localhost>
Message: Add worldelldd f

commit 7f26238948bee5d2d290fa775a78b3307ab1556cc33f2c391cd620baa38a7402
Author: PES User <pes@localhost>
Message: Initial commit
```

OPERATING SYSTEMS ORANGE PROBLEM

4B

```
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/01/eadfeac178217e2db9a7b399a62a3da57bb9ac313a1032f8a637fed07ec47a
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/70/db70c034ff7f8c0e6ae94cc8e54da0e448bd22f6f96bf4e18c15d5fd3a2f5d
.pes/objects/7f/26238948bee5d2d290fa775a78b3307ab1556cc33f2c391cd620baa38a7402
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$
```

4C

```
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ cat .pes/refs/heads/main
70db70c034ff7f8c0e6ae94cc8e54da0e448bd22f6f96bf4e18c15d5fd3a2f5d
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$
```

FINAL

```
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$ ./test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
added file.txt
added hello.txt
Status after add:
Staged changes:
  staged: file.txt
  staged: hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)

--- First Commit ---
Committed as f6a1e0fe667010911f94f2fb0be6e9787aa4b6a6b97365167421395253febfeb

Log after first commit:
commit f6a1e0fe667010911f94f2fb0be6e9787aa4b6a6b97365167421395253febfeb
Author: PES User <pes@localhost>
Message: Initial committial commiQ
```

OPERATING SYSTEMS ORANGE PROBLEM

```
--- Second Commit ---
added file.txt
Committed as d638e92f3aab4180a2e3559882d140381b333d95689c731061865c311b4ae36f

--- Third Commit ---
added bye.txt
Committed as 0992d4e4168e309b58f83d4aba77b6ae09b73912f7742358df17b93292050e47

--- Full History ---
commit 0992d4e4168e309b58f83d4aba77b6ae09b73912f7742358df17b93292050e47
Author: PES User <pes@localhost>
Message: Add farewelldd f

commit d638e92f3aab4180a2e3559882d140381b333d95689c731061865c311b4ae36f
Author: PES User <pes@localhost>
Message: Update file.txtf

commit f6a1e0fe667010911f94f2fb0be6e9787aa4b6a6b97365167421395253febfeb
Author: PES User <pes@localhost>
Message: Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
0992d4e4168e309b58f83d4aba77b6ae09b73912f7742358df17b93292050e47

--- Object Store ---
Objects created:
```

```
--- Object Store ---
Objects created:
8
.pes/objects/09/92d4e4168e309b58f83d4aba77b6ae09b73912f7742358df17b93292050e47
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d6/38e92f3aab4180a2e3559882d140381b333d95689c731061865c311b4ae36f
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f6/a1e0fe667010911f94f2fb0be6e9787aa4b6a6b97365167421395253febfeb

=== All integration tests completed ===
sharvanee@sharvanee-VirtualBox:~/PES2UG24CS459-pes-vcs$
```

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

A) To implement pes checkout <branch>, the HEAD file must be updated to point to the selected branch. The commit hash from .pes/refs/heads/<branch> is read, and the

corresponding commit and tree objects are loaded. The working directory is updated by deleting current tracked files and recreating files from the tree using blob objects. This operation is complex because it must maintain consistency between working directory, index, and commits, and must avoid overwriting uncommitted changes.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.
A) For each tracked file, compare the working directory file with the index using hashes. If they differ, the file is modified. Then compare the index version with the version in the target commit tree. If both are different, it indicates a conflict. In this case, checkout must be refused to prevent loss of uncommitted changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?
A) In detached HEAD state, new commits are created but no branch points to them. These commits are not part of any branch and may become unreachable. A user can recover them by creating a new branch pointing to that commit hash or updating a branch reference to include them.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.
A) Start from all branch heads and traverse all commits, trees, and blobs, marking each visited object as reachable using a hash set. After traversal, iterate through all objects in .pes/objects and delete those not in the reachable set. For 100,000 commits and 50 branches, around 200,000 to 500,000 objects may need to be visited depending on the number of files.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?
A) During commit creation, objects may be written but not yet referenced by HEAD. If garbage collection runs at this time, it may treat these objects as unreachable and delete them, causing corruption. Git avoids this by writing objects before updating references, using atomic operations, temporary files, and ensuring GC does not remove recently created objects.